CYLIXLAB · ASSESSMENT

Module 01

Foundations of AI & Machine Learning Algorithms

MCQ Assessment | 30 scenario-based questions with green-check marked answers and in-depth explanations

Assessment overview

Part	Questions	Topic
A	Q1–Q8	Supervised learning, generalization, regularization, leakage
B	Q9–Q14	Unsupervised learning: clustering, PCA, silhouette
C	Q15–Q22	Optimization: gradient descent, momentum, Adam / AdamW
D	Q23–Q30	Evaluation: precision, recall, F1, ROC-AUC, thresholds

HOW TO USE THIS ASSESSMENT

1. Cover the answers and choose one option per question (about 90 seconds each).
2. Check your answer; the correct option is highlighted and marked with a green check.
3. Read **every** explanation, including for questions you got right: the “why the others fail” notes target common misconceptions.
4. Score yourself with the answer key on the last page and revisit the matching theory section.

A Supervised Learning

Q1 SUPERVISED

A hospital has five years of patient records, each labelled **“readmitted within 30 days: yes / no”**. It wants to predict readmission risk for newly discharged patients. Which learning setup fits?

- **A.** Unsupervised clustering of patients
- ✓ **B. Supervised binary classification**
- **C.** Reinforcement learning with a reward for each discharge
- **D.** Dimensionality reduction with PCA

Correct answer: B

Why it is correct. Every training example has an input (patient features) and a known categorical label (yes/no), and the goal is to predict that label for new inputs. That is the definition of **supervised classification**; since there are two classes, it is binary.

Why the others fail.

✗ **A.** Clustering ignores labels. It would discover groups, but would not directly predict readmission and wastes the available labels.

✗ **C.** There is no agent interacting with an environment and receiving rewards; the data is a fixed labelled dataset.

✗ **D.** PCA compresses features; it is a preprocessing step, not a predictive model by itself.

Q2

SUPERVISED

A real-estate startup wants to predict the **sale price in dollars** of a home from its area, number of bedrooms and neighbourhood. Which formulation and loss are most appropriate?

- **A.** Binary classification trained with cross-entropy
- **B.** Clustering evaluated with the silhouette score
- **C.** Classification evaluated with accuracy
- ✓ **D. Regression trained with Mean Squared Error (or MAE)**

Correct answer: D

Why it is correct. The target is a **continuous number**, so this is a regression problem. MSE (or the outlier-robust MAE) measures how far predicted prices are from true prices and is the standard training loss.

Why the others fail.

✗ **A.** Cross-entropy is for categorical targets; price is not a class label.
✗ **B.** Clustering is unsupervised and does not predict a numeric target.
✗ **C.** Accuracy counts exact class matches; a predicted price will almost never equal the true price exactly, so accuracy is meaningless for regression.

Q3

SUPERVISED

A model reaches **99% training accuracy** but only **72% validation accuracy**, and 5-fold cross-validation confirms the gap. What is the best next step?

- ✓ **A. Reduce variance: add regularization, get more data, or simplify the model**
- **B.** Increase model complexity so it fits even better
- **C.** Train longer with a higher learning rate
- **D.** Keep checking the test set until the score rises

Correct answer: A

Why it is correct. A large train–validation gap is the signature of **overfitting (high variance)**: the model memorized training data. Regularization (L1/L2, dropout), more data, early stopping or a simpler model all reduce variance and narrow the gap.

Why the others fail.

✗ **B.** More complexity worsens overfitting; the model already fits training data nearly perfectly.
✗ **C.** Longer training with a larger step makes memorization more likely and can destabilize optimization.
✗ **D.** Repeatedly tuning against the test set leaks test information and destroys its role as an unbiased estimate.

Q4 SUPERVISED

A data scientist fits a plain linear regression to data that is clearly curved. Training $R^2 = 0.41$ and validation $R^2 = 0.40$. How should this be diagnosed and fixed?

- **A.** Overfitting: add stronger L2 regularization
- **B.** Data leakage: re-split the dataset
- ✓ **C. Underfitting (high bias): add non-linear/polynomial features or a more flexible model**
- **D.** High variance: collect ten times more data and keep the same model

Correct answer: C

Why it is correct. Both errors are poor and almost equal, so the model is too simple to capture the pattern: **high bias / underfitting**. The remedy is more capacity (polynomial terms, interactions, trees, a neural network) or better features.

Why the others fail.
✗ **A.** Regularization shrinks capacity further, which would increase bias rather than fix it.
✗ **B.** Leakage produces *suspiciously good* validation scores, not uniformly poor ones.
✗ **D.** More data does not help a model that is structurally unable to represent the relationship; bias, not variance, is the problem.

Q5 SUPERVISED

A spam detector built with **logistic regression** outputs **0.82** for a new email, and the decision threshold is 0.5. What does this mean?

- **A.** 82% of the words in the email are spam words
- ✓ **B. The model estimates an 82% probability the email is spam, so it is classified as spam**
- **C.** The model is 82% accurate overall
- **D.** The email is classified as not spam because 0.82 exceeds the threshold

Correct answer: B

Why it is correct. Logistic regression passes a linear score through the sigmoid to produce $\mathbf{P(y = 1 \mid x)}$. A value of 0.82 means an estimated 82% chance of the positive class (spam). Because $0.82 \geq 0.5$, the email is labelled spam.

Why the others fail.
✗ **A.** The output is a probability for the whole email, not a fraction of words.
✗ **C.** Accuracy is a property of the model over a dataset, not of a single prediction.
✗ **D.** Exceeding the threshold means the *positive* class is predicted; here that is spam, so this reading reverses the decision rule.

Q6 SUPERVISED

A genomics team has **500 patients** and **2,000 gene-expression features**; they believe only about 20 genes matter. They want an interpretable model that **automatically selects features**. Which approach fits best?

- ✓ **A. L1-regularized (Lasso) regression**
- **B.** L2-regularized (Ridge) regression
- **C.** Ordinary least squares via the normal equation
- **D.** k-NN with k = 1

Correct answer: A

Why it is correct. The L1 penalty $\lambda\Sigma|w_j|$ has a corner at zero that drives many coefficients **exactly to zero**, performing embedded feature selection and producing a sparse, interpretable model.

Why the others fail.

✗ **B.** Ridge shrinks weights toward zero but rarely makes them exactly zero, so all 2,000 features stay in the model.

✗ **C.** With $p > n$ the matrix $X^\top X$ is singular, so plain least squares has no unique solution and would badly overfit.

✗ **D.** 1-NN memorizes the data, gives no feature selection or interpretability, and suffers in high dimensions.

Q7 SUPERVISED

A k-NN loan-default model uses **age (20–60)** and **annual income (20,000–200,000)** as raw inputs. Predictions seem to ignore age completely. Why, and what is the fix?

- **A.** k-NN cannot handle numeric features; one-hot encode them
- **B.** Age must be removed because it is highly correlated with income
- **C.** k must be raised to about 1,000
- ✓ **D. Income dominates the Euclidean distance because of its scale; standardize the features**

Correct answer: D

Why it is correct. k-NN relies on distances. A difference of 50,000 in income dwarfs a difference of 30 years in age, so age is effectively ignored. **Standardizing or min-max scaling** puts features on comparable scales.

Why the others fail.

✗ **A.** k-NN works natively with numeric features; one-hot encoding is for categorical ones.

✗ **B.** Correlation is not the issue; scale is. Dropping age would discard useful information.

✗ **C.** A huge k smooths predictions toward the majority class but does not repair the distorted distance metric.

Q8 SUPERVISED

An analyst standardizes the **entire dataset** (mean and std computed on all rows), then splits into train/test and reports 94% test accuracy. In production the model scores 85%. What is the most likely cause?

- **A.** Underfitting caused by standardization
- **B.** The test set was too large
- ✓ **C. Data leakage: test-set statistics influenced preprocessing, inflating the test score**
- **D.** The learning rate was too high

Correct answer: C

Why it is correct. Computing the mean and std on all rows lets information from the test set leak into the training pipeline, giving an **optimistic estimate**. Fit the scaler on the training split only and apply it to the test data, ideally inside a pipeline.

Why the others fail.

✗ **A.** Standardization does not reduce model capacity; it normally helps optimization.
✗ **B.** A larger test set gives a *more reliable* estimate; it does not inflate it.
✗ **D.** A bad learning rate would hurt offline performance too, not create a gap between test and production only.

B Unsupervised Learning

Q9 UNSUPERVISED

A retailer has purchase histories for 200,000 customers but **no labels**. Marketing wants natural customer segments. Which approach is appropriate?

- ✓ **A. Cluster the standardized features with k-means or hierarchical clustering**
- **B.** Train a logistic regression classifier
- **C.** Fit a linear regression to predict spend
- **D.** Train a decision tree to predict the segment name

Correct answer: A

Why it is correct. With no labels and a goal of discovering groups, **clustering** is the right tool. Standardizing first keeps one large-scale feature (e.g. total spend) from dominating the distances.

Why the others fail.

✗ **B.** Logistic regression needs labelled classes, which do not exist here.
✗ **C.** Regression predicts a numeric target; it does not discover segments.
✗ **D.** A tree needs segment labels as targets; creating them is exactly what clustering would do.

Q10 UNSUPERVISED

Running k-means for k = 1 to 6 gives these inertia (within-cluster SSE) values: **1000, 520, 280, 250, 238, 230**. Using the elbow method, which k is best?

- **A.** k = 2
- ✓ **B. k = 3**
- **C.** k = 5
- **D.** k = 6

Correct answer: B

Why it is correct. The drops are 480, 240, 30, 12, 8. The big gains stop after k = 3 (280 → 250 is only 30), so the curve bends sharply there. Beyond the elbow, extra clusters buy very little reduction in error.

Why the others fail.

✗ **A.** At k = 2 the next step still reduces inertia by 240, so significant structure is left unexplained.
✗ **C.** Going from 3 to 5 reduces inertia only slightly; extra clusters mostly split existing groups.
✗ **D.** Inertia always decreases with k (it is 0 when k = n), so the smallest value is never the right criterion.

Q11 UNSUPERVISED

GPS pings form **two crescent-shaped dense regions** plus scattered stray points. K-means cuts the crescents in half. Which algorithm is better suited?

- **A.** K-means with a much larger k
- **B.** PCA projected to one dimension
- ✓ **C. DBSCAN**
- **D.** Gaussian mixture model with k = 2

Correct answer: C

Why it is correct. DBSCAN groups points by **density**, so it can follow arbitrary shapes such as crescents and labels sparse stragglers as **noise** instead of forcing them into clusters.

Why the others fail.

✗ **A.** A larger k only chops the shapes into more spherical pieces; it never recovers the true crescents.
✗ **B.** Projecting to 1-D collapses the geometry and would merge the structures.
✗ **D.** A GMM models each cluster as an ellipse, which still cannot represent a curved crescent well.

Q12 UNSUPERVISED

A sensor system has **50 correlated features**. PCA shows that the first 5 principal components explain a cumulative **95%** of the variance. What is the sensible conclusion?

- **A.** Delete 45 of the original sensors permanently
- **B.** The 5 components are exactly the 5 most important original sensors
- **C.** PCA must be rerun with labels to be valid
- ✓ **D. Project onto 5 components, keeping about 95% of the information in 90% fewer dimensions**

Correct answer: D

Why it is correct. Principal components are linear combinations of all original features ordered by variance. Keeping the top 5 compresses 50 dimensions to 5 while preserving roughly 95% of the variance, which is a common way to reduce noise and cost.

Why the others fail.

✗ **A.** Components are new axes, not selections of original features; the hardware decision does not follow from PCA directly.

✗ **B.** Each component mixes many original sensors, so it does not map to one sensor.

✗ **C.** PCA is unsupervised; it never needs labels (standardizing the features first is the real requirement).

Q13 UNSUPERVISED

For one data point, the mean distance to the other points in its own cluster is **a = 2**, and the mean distance to the nearest other cluster is **b = 6**. What is its silhouette value and meaning?

- **A.** ≈ 0.33: ambiguous
- ✓ **B. ≈ 0.67: well matched to its own cluster**
- **C.** 3.00: extremely well clustered
- **D.** −0.67: probably in the wrong cluster

Correct answer: B

Why it is correct. s = (b − a) / max(a, b) = (6 − 2) / 6 = **0.667**. A value near +1 means the point is much closer to its own cluster than to any neighbouring cluster.

Why the others fail.

✗ **A.** 0.33 comes from a/b, which is not the silhouette formula.

✗ **C.** Silhouette values lie in [−1, 1]; 3.00 (b/a) is impossible.

✗ **D.** A negative value would require a > b, i.e. the point is nearer another cluster, which is the opposite of this case.

Q14 UNSUPERVISED

Running k-means three times on the same data gives **different final clusters and different inertia** each time. What is the recommended remedy?

- ✓ **A. Use k-means++ initialization and several restarts (n_init), keeping the run with the lowest inertia**
- ○ **B.** Switch to a supervised classifier
- ○ **C.** Raise the learning rate
- ○ **D.** Skip standardization to make results stable

Correct answer: A

Why it is correct. K-means converges to a **local** optimum that depends on the random starting centroids. k-means++ spreads initial centroids out, and multiple restarts let you keep the best solution.

Why the others fail.

✗ **B.** No labels exist, so a supervised model is not an option.
✗ **C.** K-means has no learning rate; it alternates assignment and mean updates.
✗ **D.** Removing standardization usually makes clustering *worse* and does not address initialization.

C Optimization

Q15 OPTIMIZATION

A network trains normally with learning rate 0.001. After changing it to **1.0**, the loss reads 2.3, 8.7, 54.1, then NaN. What is the problem and the fix?

- ○ **A.** The model is underfitting; add more layers
- ○ **B.** The batch size is too large; use batch size 1
- ✓ **C. The learning rate is too large and training diverges; lower it (and consider warm-up or gradient clipping)**
- ○ **D.** The model is overfitting; add dropout

Correct answer: C

Why it is correct. Each update overshoots the minimum by more than it corrects, so the loss grows exponentially until it overflows. The fix is a smaller η, possibly with **warm-up** and **gradient clipping**.

Why the others fail.

✗ **A.** Underfitting shows a high but *stable* loss, not an explosion.
✗ **B.** Batch size 1 adds noise and would not cure an overshoot caused by the step size.
✗ **D.** Overfitting appears as a widening train–validation gap, not a divergent training loss.

Q16 OPTIMIZATION

For $L(w) = w^2$ the gradient is $2w$. Starting at **$w = 4$** with learning rate **$\eta = 0.1$**, what is w after **one** gradient-descent step?

- **A.** 3.6
- **B.** 3.8
- **C.** 4.8
- **D. 3.2** ✓

Correct answer: D

Why it is correct. $w_1 = w_0 - \eta \cdot \nabla L(w_0) = 4 - 0.1 \cdot (2 \cdot 4) = 4 - 0.8 = \mathbf{3.2}$. Every step multiplies w by $(1 - 2\eta) = 0.8$.

Why the others fail.

✗ **A.** 3.6 uses a gradient of 4 (w itself) instead of $2w = 8$.
✗ **B.** 3.8 subtracts $\eta \cdot 2$ and forgets that the gradient depends on w.
✗ **C.** 4.8 adds the step, moving uphill; gradient *descent* subtracts the gradient.

Q17 OPTIMIZATION

A team must train on **10 million images** that do not fit in memory at once. Which optimization strategy is the practical choice?

- **A.** Full-batch gradient descent
- **B. Mini-batch stochastic gradient descent** ✓
- **C.** Solving the normal equation
- **D.** Computing one exact gradient on the whole dataset each step, but with Adam

Correct answer: B

Why it is correct. Mini-batch SGD estimates the gradient from a small random subset (e.g. 256 images), so memory use is bounded, updates are frequent and GPUs are used efficiently. It is the default in deep learning.

Why the others fail.

✗ **A.** Full-batch GD needs every sample for each update, which is infeasible when data does not fit in memory.
✗ **C.** The normal equation applies to linear models and costs roughly $O(p^3)$ in the feature count.
✗ **D.** Adam still needs a gradient; computing it on the full data has the same memory problem.

Q18 OPTIMIZATION

Training oscillates across the steep walls of a long, narrow valley in the loss surface and moves slowly along the valley floor. Which change addresses this directly?

- ✓ **A. Add momentum (or switch to Adam)**
- **B.** Decrease the number of epochs
- **C.** Remove the bias terms
- **D.** Use a constant, much larger learning rate

Correct answer: A

Why it is correct. Momentum accumulates an exponentially-weighted average of gradients: the oscillating components across the walls cancel, while the consistent component along the valley adds up, **speeding progress**.

Why the others fail.

✗ **B.** Fewer epochs gives less time to converge and does nothing about oscillation.
✗ **C.** Bias terms are unrelated to the ill-conditioned geometry.
✗ **D.** A bigger step would intensify the oscillation across the steep direction and risk divergence.

Q19 OPTIMIZATION

A language model has embeddings for very **rare words** that receive tiny, infrequent gradients, and plain SGD with one global learning rate trains them very slowly. Which property of Adam helps most?

- **A.** Using the full dataset for each update
- **B.** Forcing all weights to have the same step length
- ✓ **C. Per-parameter step sizes from the second-moment estimate v_t (bigger steps for small, rare gradients)**
- **D.** Removing the need for any learning rate

Correct answer: C

Why it is correct. Adam divides the update by $\sqrt{\hat{v}_t}$, the running root-mean-square of each parameter's gradients. Parameters with small or sparse gradients therefore get **relatively larger** steps, which suits embeddings and sparse features.

Why the others fail.

✗ **A.** Adam is still typically run on mini-batches.
✗ **B.** Adam does the opposite of forcing equal step lengths: it adapts them per parameter.
✗ **D.** Adam still has a learning rate η (default 0.001), it is just less sensitive to its exact value.

Q20 OPTIMIZATION

In Adam, at step t = 1 with $\mathbf{m_0 = 0}$, $\beta_1 = 0.9$ and gradient $\mathbf{g_1 = 10}$, what is the bias-corrected first moment $\hat{\mathbf{m}}_1$?

- **A.** 1
- **B.** 0.1
- **C.** 9
- **D. 10**

Correct answer: D

Why it is correct. $m_1 = 0.9 \cdot 0 + 0.1 \cdot 10 = 1$ (biased toward zero because of the zero initialization). Dividing by $(1 - \beta_1^1) = 0.1$ gives $\hat{m}_1 = \mathbf{10}$, which equals the gradient.

Why the others fail.

✗ **A.** 1 is the *uncorrected* m_1, the very bias the correction removes.
✗ **B.** 0.1 is the weight $(1 - \beta_1)$ alone, not the estimate.
✗ **C.** 9 is $\beta_1 \cdot 10$, which would be the contribution of a previous average, not the corrected estimate.

Q21 OPTIMIZATION

At the first Adam step the gradient is $\mathbf{g = 2}$. With $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and ε negligible, by roughly how much does the parameter change?

- **A. About 0.001 (decreasing it by η)**
- **B.** About 0.002
- **C.** About 0.0005
- **D.** About 0.004

Correct answer: A

Why it is correct. $\hat{m}_1 = g = 2$ and $\hat{v}_1 = g^2 = 4$ after bias correction, so the step is $\eta \cdot 2/\sqrt{4} = \eta = \mathbf{0.001}$. On the first step Adam moves each parameter by about η in the direction of $-\text{sign}(g)$.

Why the others fail.

✗ **B.** 0.002 would be $\eta \cdot g$, as in plain GD, and ignores the division by $\sqrt{\hat{v}}$.
✗ **C.** 0.0005 would result from dividing by g twice or from skipping bias correction in a way that does not match the equations.
✗ **D.** 0.004 would be $\eta \cdot g^2$; Adam divides by the gradient scale rather than multiplying by it.

Q22 OPTIMIZATION

A team trains with Adam and an L2 penalty added to the loss, but it generalizes worse than expected. A practitioner suggests a change to the optimizer. Which is best?

- **A.** Remove all regularization
- ✓ **B. Switch to AdamW, which decouples weight decay from the adaptive gradient update**
- **C.** Set β_2 to 0
- **D.** Replace the loss with accuracy

Correct answer: B

Why it is correct. Inside Adam the L2 gradient is divided by $\sqrt{\hat{v}}$ like any other gradient, so its shrinking effect becomes uneven across parameters. **AdamW** applies weight decay directly to the weights, giving the intended regularization and usually better generalization.

Why the others fail.

✗ **A.** Removing regularization generally increases overfitting.
✗ **C.** $\beta_2 = 0$ removes the smoothing of squared gradients and makes updates extremely noisy.
✗ **D.** Accuracy is not differentiable, so it cannot be optimized with gradient-based methods.

D Evaluation Metrics

Q23 METRICS

A classifier on 200 samples gives **TP = 40, FP = 10, FN = 20, TN = 130**. What are its precision and recall?

- **A.** Precision ≈ 0.67, Recall = 0.80
- **B.** Precision = 0.85, Recall = 0.85
- ✓ **C. Precision = 0.80, Recall ≈ 0.67**
- **D.** Precision = 0.80, Recall = 0.80

Correct answer: C

Why it is correct. Precision = TP/(TP + FP) = 40/50 = **0.80**. Recall = TP/(TP + FN) = 40/60 = **0.667**. (For reference, accuracy = 170/200 = 0.85 and F1 ≈ 0.727.)

Why the others fail.

✗ **A.** This swaps the two definitions: precision uses predicted positives in the denominator, recall uses actual positives.
✗ **B.** 0.85 is the *accuracy*; precision and recall are not equal to it.
✗ **D.** Recall must account for the 20 missed positives (FN), so it cannot equal precision here.

Q24 METRICS

A **cancer-screening** test is reviewed. Missing a real case could be fatal, while a false alarm only leads to a follow-up test. Which metric/adjustment should be prioritized?

- **A.** Precision; raise the decision threshold
- **B.** Accuracy; leave the threshold at 0.5
- **C.** Specificity only
- ✓ **D. Recall (sensitivity); lower the decision threshold**

Correct answer: D

Why it is correct. The expensive error is a **false negative**, and recall = TP/(TP + FN) measures exactly how many real cases are caught. Lowering the threshold flags more patients, increasing recall at the cost of more (tolerable) false positives.

Why the others fail.

✗ **A.** Raising the threshold increases precision but misses more real cases, the wrong direction here.
✗ **B.** Accuracy treats both errors alike and is dominated by the many healthy patients.
✗ **C.** Specificity measures correctly cleared healthy people; it says nothing about missed cases.

Q25 METRICS

An email service auto-moves messages to a **spam folder users rarely check**. Wrongly hiding a legitimate email is much worse than letting some spam through. Which approach follows?

- **A.** Optimize recall and lower the threshold
- ✓ **B. Optimize precision and raise the decision threshold**
- **C.** Use accuracy at the default threshold
- **D.** Maximize the number of predicted positives

Correct answer: B

Why it is correct. A legitimate email in the spam folder is a **false positive**. Precision = TP/(TP + FP) penalizes exactly this, and a higher threshold flags only emails the model is very sure about.

Why the others fail.

✗ **A.** High recall at all costs would flag more legitimate mail as spam.
✗ **C.** Accuracy hides the cost asymmetry between the two error types.
✗ **D.** Predicting more positives increases false positives, the very error to avoid.

Q26 METRICS

Only **1% of transactions are fraud**. A model predicts “not fraud” for everything and reports 99% accuracy. Which statement exposes the real problem?

- ✓ **A. Its recall for fraud is 0%, so it catches no fraud at all**
- **B.** 99% accuracy proves the model is excellent
- **C.** Its specificity of 100% shows it works well
- **D.** Its F1 score must therefore be about 0.99

Correct answer: A

Why it is correct. This is the **accuracy paradox**. All 99% of correct predictions are the negative class; TP = 0, so recall = 0 (and F1 = 0). Recall, F1 or PR-AUC reveal the failure that accuracy hides.

Why the others fail.

✗ **B.** High accuracy on a 99/1 split is achievable by ignoring the minority class entirely.
✗ **C.** Specificity only measures how well negatives are cleared; the model fails on the class that matters.
✗ **D.** With TP = 0, precision and recall are 0 (or undefined), so F1 is 0, not 0.99.

Q27 METRICS

A model has **precision = 0.90** and **recall = 0.50**. What is its F1 score?

- **A.** 0.70
- **B.** 0.45
- **C.** 0.50
- ✓ **D. ≈ 0.64**

Correct answer: D

Why it is correct. $F1 = 2PR/(P + R) = 2 \cdot 0.9 \cdot 0.5/1.4 = 0.9/1.4 \approx$ **0.643**. As a harmonic mean, F1 is pulled toward the smaller of the two values.

Why the others fail.

✗ **A.** 0.70 is the arithmetic mean, which overstates performance when P and R differ greatly.
✗ **B.** 0.45 is just the product P·R.
✗ **C.** 0.50 equals the smaller value; the harmonic mean is slightly above the minimum, not equal to it.

Q28 METRICS

A binary classifier achieves a **ROC-AUC of 0.50** on a validation set. What does this indicate?

- **A.** It is correct half of the time but otherwise useful
- **B.** It is a perfect classifier
- ✓ **C. Its ranking ability is no better than random guessing**
- **D.** Its precision and recall are both exactly 0.5

Correct answer: C

Why it is correct. AUC is the probability that a random positive is scored higher than a random negative. At **0.5** that is a coin flip: the scores carry no discriminative information (the ROC curve lies on the diagonal).

Why the others fail.

✗ **A.** AUC is not an accuracy; 0.5 means *no* ranking skill.
✗ **B.** A perfect classifier has AUC = 1.0.
✗ **D.** AUC is threshold-free and says nothing specific about precision or recall values at one threshold.

Q29 METRICS

A rare-disease model (**0.1% positives**) shows ROC-AUC = 0.97, yet at 80% recall its precision is only about 4%. Which evaluation view gives the more honest picture for this class imbalance?

- ✓ **A. The Precision–Recall curve / Average Precision**
- **B.** The ROC curve alone
- **C.** Training accuracy
- **D.** The mean of TPR and FPR

Correct answer: A

Why it is correct. With so many true negatives, FPR stays tiny even when false positives swamp true positives, making ROC look rosy. The **PR curve** uses precision, which directly shows that most alerts are false, so it reflects the practical situation.

Why the others fail.

✗ **B.** ROC-AUC is exactly the number that looks over-optimistic here.
✗ **C.** Training accuracy is dominated by negatives and is measured on seen data.
✗ **D.** TPR and FPR are on different scales; their average is not a standard or meaningful metric.

Q30 METRICS

A fraud model’s threshold is lowered from **0.5 to 0.2**. What is the typical effect?

- **A.** Precision and recall both increase
- **B. Recall and FPR increase; precision usually decreases**
- **C.** Recall decreases and precision increases
- **D.** Nothing changes because AUC is fixed

Correct answer: B

Why it is correct. A lower threshold labels more cases positive: it catches more real fraud (recall up), but also flags more legitimate transactions (FP up, FPR up), which usually lowers precision. You move *along* the ROC curve.

Why the others fail.

✗ **A.** Precision and recall generally trade off; both rarely rise together just by changing the threshold.
✗ **C.** That describes *raising* the threshold.
✗ **D.** AUC summarizes all thresholds and stays the same, but the operating point (precision, recall) moves.

✓ Answer Key & Score Sheet

Q	Ans	Topic	Q	Ans	Topic
1	**B**	Supervised	16	**D**	Optimization
2	**D**	Supervised	17	**B**	Optimization
3	**A**	Supervised	18	**A**	Optimization
4	**C**	Supervised	19	**C**	Optimization
5	**B**	Supervised	20	**D**	Optimization
6	**A**	Supervised	21	**A**	Optimization
7	**D**	Supervised	22	**B**	Optimization
8	**C**	Supervised	23	**C**	Metrics
9	**A**	Unsupervised	24	**D**	Metrics
10	**B**	Unsupervised	25	**B**	Metrics
11	**C**	Unsupervised	26	**A**	Metrics
12	**D**	Unsupervised	27	**D**	Metrics
13	**B**	Unsupervised	28	**C**	Metrics
14	**A**	Unsupervised	29	**A**	Metrics
15	**C**	Optimization	30	**B**	Metrics

Score interpretation

Score	Level	Recommended action
26–30	Mastery	Move on to Module 02; try explaining each answer aloud.
20–25	Proficient	Review the sections behind your misses, then redo those questions.
13–19	Developing	Re-read the Theory Study Guide sections, work the numeric examples by hand, retake after 48 hours.
0–12	Foundation	Restart with Section 1 and 2 of the guide, then attempt the quiz again.

KEY IDEA
Track your misses by topic. If you missed 2+ questions in one part (A–D), that part is your priority for revision. Numeric questions (Q13, Q16, Q20, Q21, Q23, Q27) can all be solved with the formula sheet in Section 6 of the Study Guide.